

Dr. Sanjay Adiwali

Joint Director

Centre for Development of Advanced Computing (C-DAC)

Electronics City, Bangalore 560 100

Web Application Security

Outline

- HTTP Overview and Evolution
- HTTP Header and Response codes
- Methods used in HTTP communications
- Web Application Security

Hypertext Transfer Protocol - HTTP

- Application Layer Protocol that supports communication between web browsers and web servers.
- Uses TCP Port 80 or 443.
- It defines how messages are formatted and transmitted between:
 - Web browsers (clients)
 - Web servers
- HTTP follows client server model and it is a request and response protocol.
- HTTP communicates with Header.

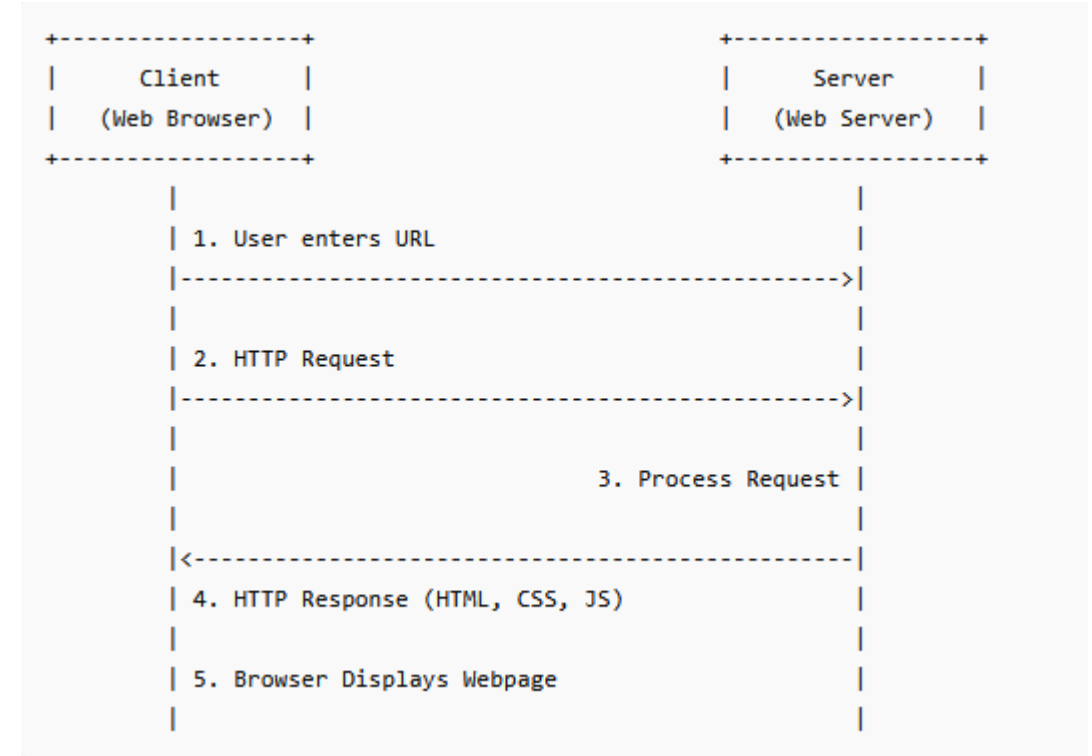
How HTTP Works

• Client-Server Model

1. User enters URL in browser
2. Browser sends HTTP request
3. Server processes request
4. Server sends HTTP response
5. Browser displays webpage

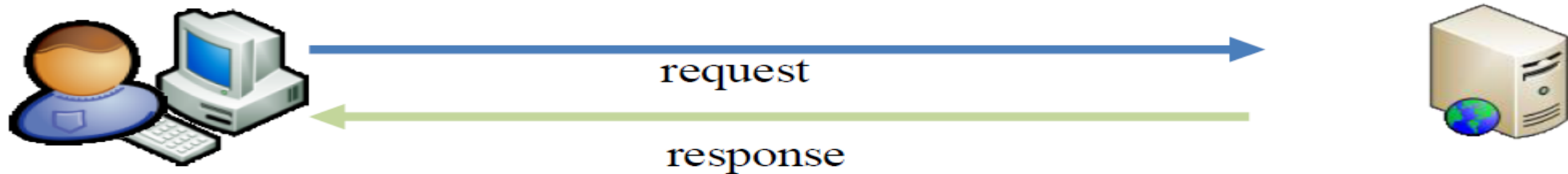
• Example:

- Client → Request: “Get homepage”
- Server → Response: HTML page



HTTP: The Request – Response Protocol

- HTTP has a simple structure:
 - Client sends a request
 - Server returns a reply (response)
- HTTP can support multiple request-reply exchanges over a single TCP connection
- Stateless Protocol
- For a web application pen tester, its very important to understand the Request and Response format which would be helpful for future testing.



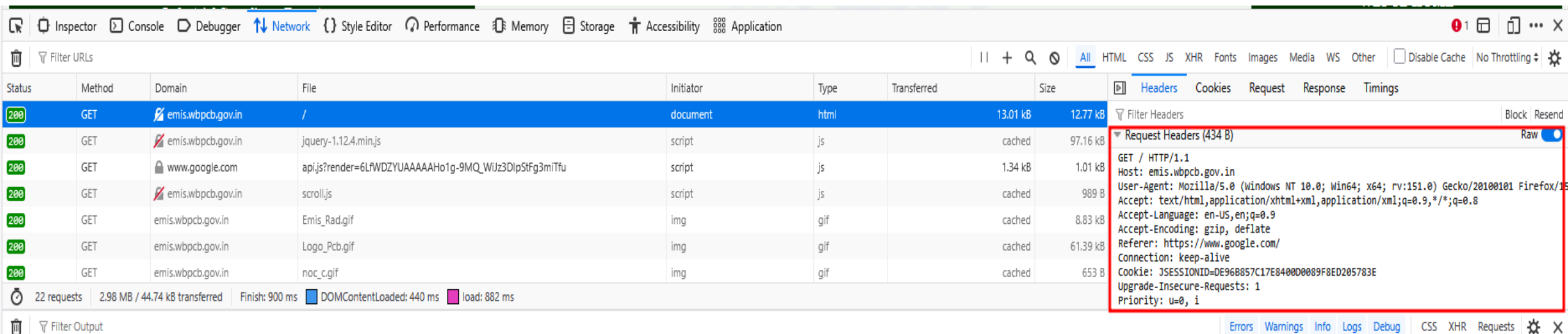
HTTP Request Structure

• Components of Request

- Request Line
- Headers
- Body (optional)

```

> Frame 6876: 391 bytes on wire (3128 bits), 391 bytes captured (3128 bits) on interface eth0, id 0
> Ethernet II, Src: PCSSystemtec_8d:f7:d3 (08:00:27:8d:f7:d3), Dst: 52:54:00:12:35:00 (52:54:00:12:35:00)
> Internet Protocol Version 4, Src: 10.0.2.15, Dst: 210.212.0.226
> Transmission Control Protocol, Src Port: 41542, Dst Port: 80, Seq: 1, Ack: 1, Len: 337
> Hypertext Transfer Protocol
  > GET / HTTP/1.1\r\n
    Host: emis.wbpcb.gov.in\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate\r\n
    Connection: keep-alive\r\n
    Upgrade-Insecure-Requests: 1\r\n
    Priority: u=0, i\r\n
    \r\n
    [Response in frame: 6921]
    [Full request URI: http://emis.wbpcb.gov.in/]
  
```



The screenshot shows the Chrome DevTools Network tab with a list of requests. The first request is highlighted, and its details are expanded in the right pane.

Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	GET	emis.wbpcb.gov.in	/	document	html	13.01 kB	12.77 kB
200	GET	emis.wbpcb.gov.in	jquery-1.12.4.min.js	script	js	cached	97.16 kB
200	GET	www.google.com	api.js?render=6LFWZUYAAAAHo1g-9MQ_Wijz3DlpStFg3miTfu	script	js	1.34 kB	1.01 kB
200	GET	emis.wbpcb.gov.in	scroll.js	script	js	cached	989 B
200	GET	emis.wbpcb.gov.in	Emis_Rad.gif	img	gif	cached	8.83 kB
200	GET	emis.wbpcb.gov.in	Logo_Pcb.gif	img	gif	cached	61.39 kB
200	GET	emis.wbpcb.gov.in	noc_c.gif	img	gif	cached	653 B

Request Headers (434 B)

```

GET / HTTP/1.1
Host: emis.wbpcb.gov.in
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:151.0) Gecko/20100101 Firefox/151.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Referer: https://www.google.com/
Connection: keep-alive
Cookie: JSESSIONID=DE96B857C17E8400D0089F8ED205783E
Upgrade-Insecure-Requests: 1
Priority: u=0, i
  
```

HTTP Response Structure

• Components of Response

- Status Line
- Headers
- Body (content)

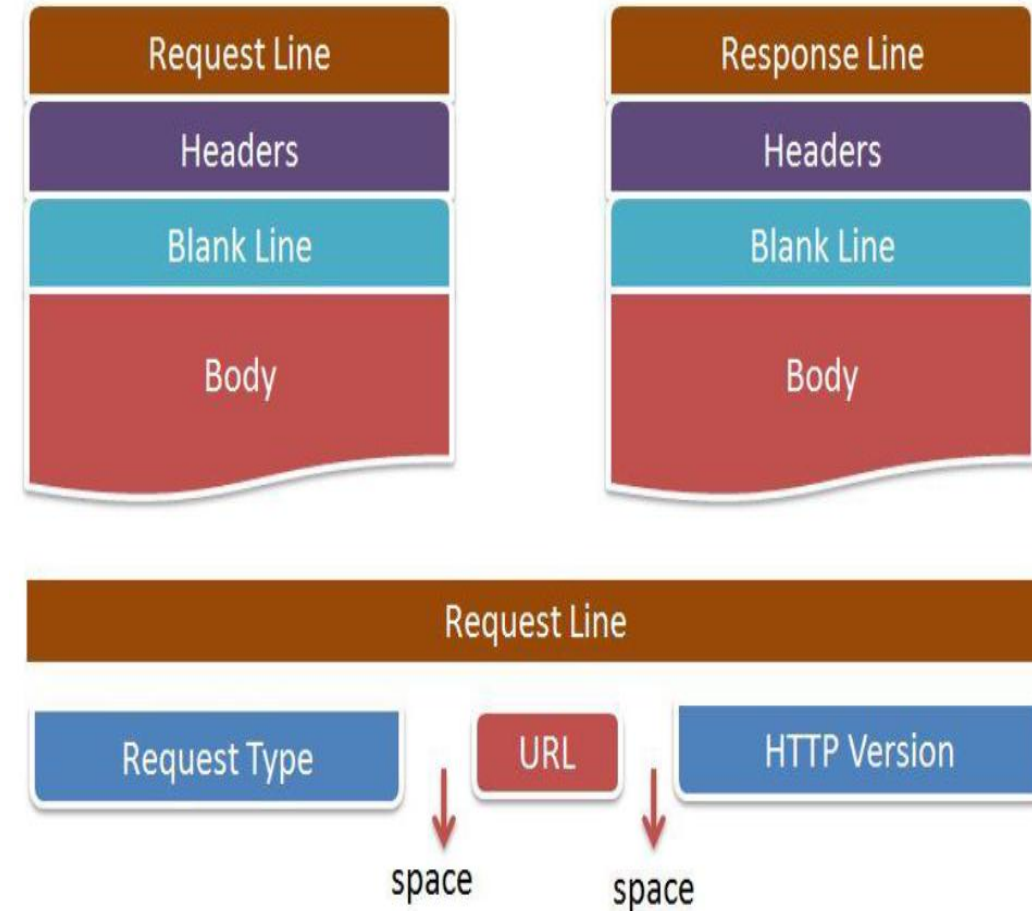
```

Frame 6921: 1428 bytes on wire (11424 bits), 1428 bytes captured (11424 bits) on interface eth0, id 0
Ethernet II, Src: 52:54:00:12:35:00 (52:54:00:12:35:00), Dst: PCSSystemtec_8d:f7:d3 (08:00:27:8d:f7:d3)
Internet Protocol Version 4, Src: 210.212.0.226, Dst: 10.0.2.15
Transmission Control Protocol, Src Port: 80, Dst Port: 41542, Seq: 11661, Ack: 338, Len: 1374
[12 Reassembled TCP Segments (13034 bytes): #6903(1440), #6904(1440), #6907(1440), #6908(1440), #6909(864), #6910(1440), #6911(1440), #6912(1440), #6913(1440), #6914(1440), #6915(1440), #6916(1440)]
Hypertext Transfer Protocol, has 3 chunks (including last chunk)
  HTTP/1.1 200 OK\r\n
  Server: Apache-Coyote/1.1\r\n
  X-Frame-Options: DENY\r\n
  Set-Cookie: JSESSIONID=80E079430383F39406F7876596B5887C; Path=/\r\n
  Content-Type: text/html; charset=ISO-8859-1\r\n
  Transfer-Encoding: chunked\r\n
  Date: Mon, 08 Jun 2026 09:08:42 GMT\r\n
  \r\n
  [Request in frame: 6876]
  [Time since request: 0.044556840 seconds]
  [Request URI: /]
  [Full request URI: http://emis.wbpcb.gov.in/]
  HTTP chunked response
  File Data: 12770 bytes
Line-based text data: text/html (282 lines)
  
```

Status	Method	Domain	File	Initiator	Type	Transferred	Size	Headers	Cookies	Request	Response	Timings
200	GET	emis.wbpcb.gov.in	/	document	html	13.01 kB	12.77 kB	Filter Headers				
200	GET	emis.wbpcb.gov.in	jquery-1.12.4.min.js	script	js	cached	97.16 kB	Status	200 OK (2)			
200	GET	www.google.com	api.js?render=6LFWZDYUAAAAHo1g-9MQ_WiJz3DlpStFg3miTfu	script	js	1.34 kB	1.01 kB	Version	HTTP/1.1			
200	GET	emis.wbpcb.gov.in	scroll.js	script	js	cached	989 B	Transferred	13.01 kB (12.77 kB size)			
200	GET	emis.wbpcb.gov.in	Emis_Rad.gif	img	gif	cached	8.83 kB	Referrer Policy	origin			
200	GET	emis.wbpcb.gov.in	Logo_Pcb.gif	img	gif	cached	61.39 kB	Request Priority	Highest			
200	GET	emis.wbpcb.gov.in	noc_c.gif	img	gif	cached	653 B	DNS Resolution	System			
200	GET	emis.wbpcb.gov.in	co_c.gif	img	gif	cached	618 B	Response Headers (243 B)				
200	GET	emis.wbpcb.gov.in	bio.gif	img	gif	cached	938 B	HTTP/1.1 200 OK				
200	GET	emis.wbpcb.gov.in	hazard.gif	img	gif	cached	924 B	Server: Apache-Coyote/1.1				
200	GET	emis.wbpcb.gov.in	muni.gif	img	gif	cached	998 B	X-Frame-Options: DENY				
								Set-Cookie: JSESSIONID=F5ED3058C290408B9CE10CA853A33524; Path=				
								Content-Type: text/html; charset=ISO-8859-1				
								Transfer-Encoding: chunked				
								Date: Mon, 08 Jun 2026 09:56:05 GMT				

HTTP Headers

- The first line in request message is called request line and the same is called as status line/response line in Response message and all others are common to both messages.
- The header exchanges additional information between the client and the server.



HTTP Headers

- HTTP headers are key-value pairs exchanged between the client and server that provide additional information about the request or response.
- Request headers
- Response Headers

HTTP Request Headers

- Sent by the client to the server.
- Common Headers

Header

Host

User-Agent

Accept

Authorization

Cookie

Purpose

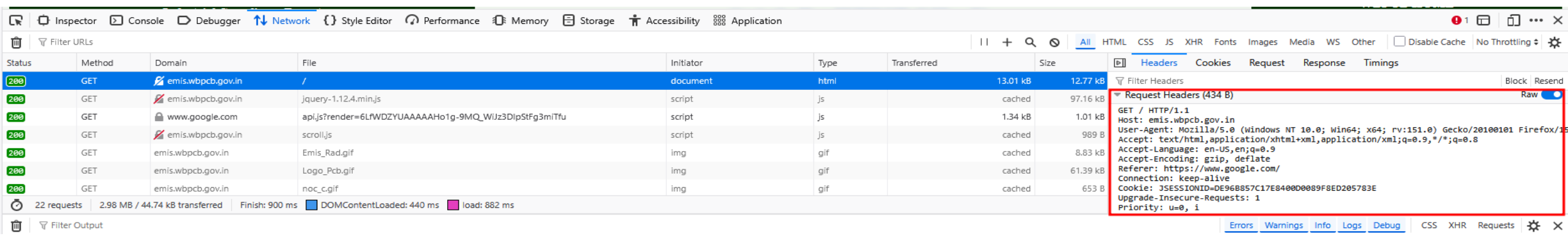
Specifies the server hostname

Identifies the client/browser

Defines acceptable content types

Sends authentication credentials

Sends stored cookies



The screenshot shows the Chrome DevTools Network tab with a list of requests to `emis.wbpcb.gov.in`. The first request is selected, and its headers are expanded, showing the following information:

```

GET / HTTP/1.1
Host: emis.wbpcb.gov.in
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:101.0) Gecko/20100101 Firefox/101.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Referer: https://www.google.com/
Connection: keep-alive
Cookie: JSESSIONID=DE96B857C17E8400D0089F8ED205783E
Upgrade-Insecure-Requests: 1
Priority: u=0, i
  
```

HTTP Response Headers

- Sent by the server to the client.
- Common response headers

Header

Purpose

Content-Type

Type of returned content

Content-Length

Size of response body

Cache-Control

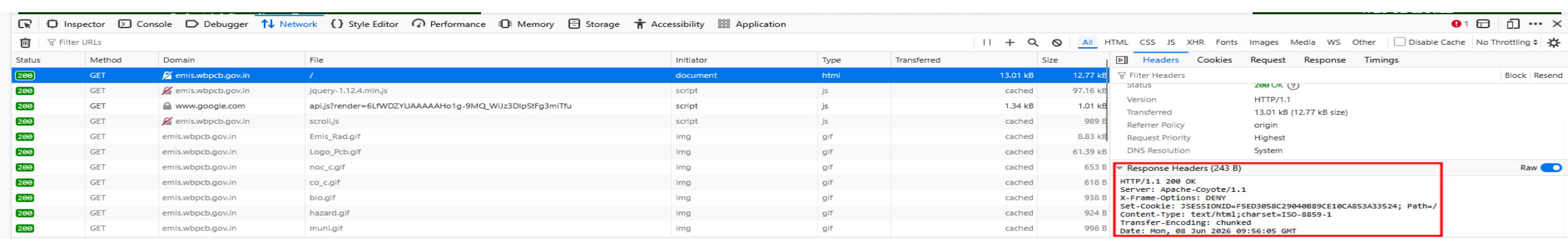
Caching instructions

Set-Cookie

Creates cookies on client

Server

Information about web server



Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	GET	emis.wbpcb.gov.in	/	document	html	13.01 kB	12.77 kB
200	GET	emis.wbpcb.gov.in	jquery-1.12.4.min.js	script	js	cached	97.16 kB
200	GET	www.google.com	api.js?render=6LFWZUYUAAAAHo1g-9MQ_WUz3DlpStFg3miTfu	script	js	1.34 kB	1.01 kB
200	GET	emis.wbpcb.gov.in	scroll.js	script	js	cached	989 B
200	GET	emis.wbpcb.gov.in	Emis_Rad.gif	img	gif	cached	8.83 kB
200	GET	emis.wbpcb.gov.in	Logo_Pcb.gif	img	gif	cached	61.39 kB
200	GET	emis.wbpcb.gov.in	noc_c.gif	img	gif	cached	653 B
200	GET	emis.wbpcb.gov.in	co_c.gif	img	gif	cached	618 B
200	GET	emis.wbpcb.gov.in	bio.gif	img	gif	cached	938 B
200	GET	emis.wbpcb.gov.in	hazard.gif	img	gif	cached	924 B
200	GET	emis.wbpcb.gov.in	muni.gif	img	gif	cached	998 B

Filter Headers

Status: 200 OK

Version: HTTP/1.1

Transferred: 13.01 kB (12.77 kB size)

Referrer Policy: origin

Request Priority: Highest

DNS Resolution: System

Response Headers (243 B)

HTTP/1.1 200 OK

Server: Apache-Coyote/1.1

X-Frame-Options: DENY

Set-Cookie: JSESSIONID=F5ED3058C29040B89CE10CA853A33524; Path=/

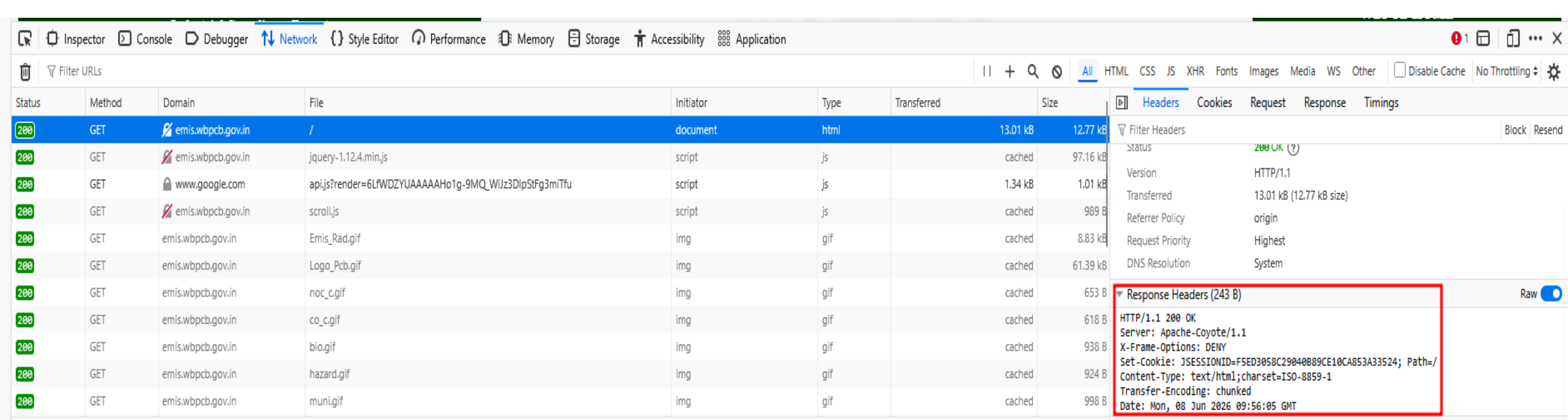
Content-Type: text/html; charset=ISO-8859-1

Transfer-Encoding: chunked

Date: Mon, 08 Jun 2026 09:56:05 GMT

HTTP Response Codes

- HTTP response codes are 3-digit numbers returned by the server
- They indicate the result of a client's request
- Helps browser understand what happened with the request



The screenshot shows the Chrome DevTools Network tab. The top toolbar includes icons for Inspector, Console, Debugger, Network, Style Editor, Performance, Memory, Storage, Accessibility, and Application. The Network tab is active, displaying a list of requests. The first request is a GET request to `emis.wbpcb.gov.in /` with a status of 200. The response headers for this request are expanded and highlighted with a red box.

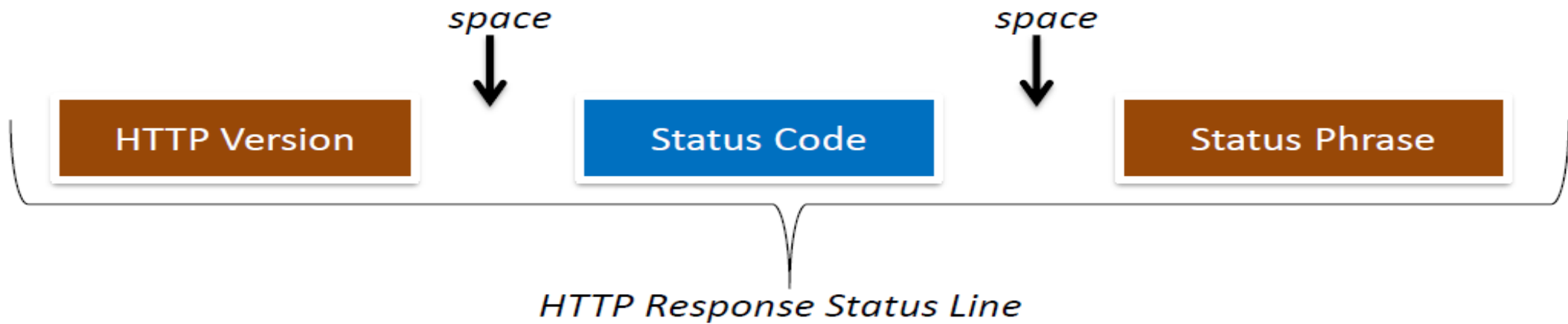
Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	GET	emis.wbpcb.gov.in	/	document	html	13.01 kB	12.77 kB
200	GET	emis.wbpcb.gov.in	jquery-1.12.4.min.js	script	js	cached	97.16 kB
200	GET	www.google.com	api.js?render=6LfWDZYUAAAAHo1g-9MQ_WiJz3DlpStFg3miTfu	script	js	1.34 kB	1.01 kB
200	GET	emis.wbpcb.gov.in	scroll.js	script	js	cached	989 B
200	GET	emis.wbpcb.gov.in	Emis_Rad.gif	img	gif	cached	8.83 kB
200	GET	emis.wbpcb.gov.in	Logo_Pcb.gif	img	gif	cached	61.39 kB
200	GET	emis.wbpcb.gov.in	noc_c.gif	img	gif	cached	653 B
200	GET	emis.wbpcb.gov.in	co_c.gif	img	gif	cached	618 B
200	GET	emis.wbpcb.gov.in	bio.gif	img	gif	cached	938 B
200	GET	emis.wbpcb.gov.in	hazard.gif	img	gif	cached	924 B
200	GET	emis.wbpcb.gov.in	muni.gif	img	gif	cached	998 B

Response Headers (243 B)

```

HTTP/1.1 200 OK
Server: Apache-Coyote/1.1
X-Frame-Options: DENY
Set-Cookie: JSESSIONID=F5ED3058C29040B89CE10CA853A33524; Path=/
Content-Type: text/html; charset=ISO-8859-1
Transfer-Encoding: chunked
Date: Mon, 08 Jun 2026 09:56:05 GMT
  
```

HTTP Response Codes



Status Code	Meaning
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client Error
5xx	Server Error

Categories of Status Codes

- HTTP status codes are divided into **5 classes**:

Code Range	Meaning
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client Error
5xx	Server Error

1xx – Informational

- Request received, processing continues
- Examples:
 - 100 Continue → Server is ready for request
 - 101 Switching Protocols → Protocol upgrade (e.g., HTTP to WebSocket)

2xx – Success

- Request was successfully processed
- Examples:
 - 200 OK → Request successful
 - 201 Created → Resource created successfully
 - 204 No Content → Request successful, no response body

3xx – Redirection

- Client must take additional action
- Examples:
 - 301 Moved Permanently → Resource moved to new URL
 - 302 Found → Temporary redirection
 - 304 Not Modified → Cached version is valid

4xx – Client Errors

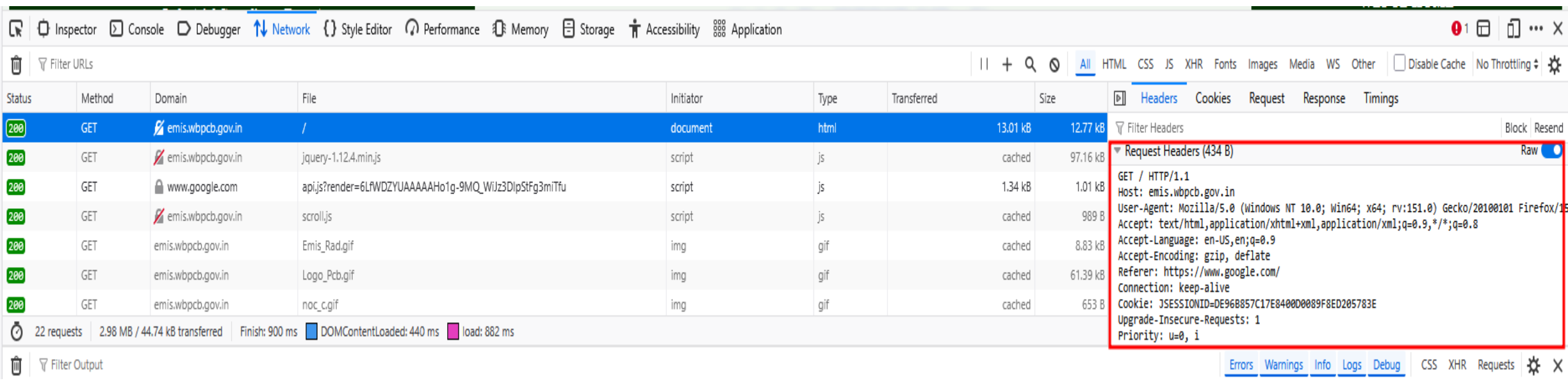
- Error caused by client request
- Examples:
 - 400 Bad Request → Invalid request syntax
 - 401 Unauthorized → Authentication required
 - 403 Forbidden → Access denied
 - 404 Not Found → Resource not found

5xx – Server Errors

- Server failed to process valid request
- Examples:
 - 500 Internal Server Error → Generic server failure
 - 502 Bad Gateway → Invalid response from upstream server
 - 503 Service Unavailable → Server overloaded or down
 - 504 Gateway Timeout → Server did not respond in time

HTTP Methods

- HTTP methods define the type of action to be performed on a resource
- They are part of the HTTP request sent by the client
- Help server understand what the client wants to do



The screenshot shows the Chrome DevTools Network tab with a list of requests. The first request is highlighted, and its headers are expanded.

Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	GET	emis.wbpcb.gov.in	/	document	html	13.01 kB	12.77 kB
200	GET	emis.wbpcb.gov.in	jquery-1.12.4.min.js	script	js	cached	97.16 kB
200	GET	www.google.com	api.js?render=6LfWDZYUAAAAHo1g-9MQ_WiJz3DlpStFg3miTfu	script	js	1.34 kB	1.01 kB
200	GET	emis.wbpcb.gov.in	scroll.js	script	js	cached	989 B
200	GET	emis.wbpcb.gov.in	Emis_Rad.gif	img	gif	cached	8.83 kB
200	GET	emis.wbpcb.gov.in	Logo_Pcb.gif	img	gif	cached	61.39 kB
200	GET	emis.wbpcb.gov.in	noc_c.gif	img	gif	cached	653 B

Request Headers (434 B)

```

GET / HTTP/1.1
Host: emis.wbpcb.gov.in
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:151.0) Gecko/20100101 Firefox/15.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Referer: https://www.google.com/
Connection: keep-alive
Cookie: JSESSIONID=DE96B857C17E8400D0089F8ED205783E
Upgrade-Insecure-Requests: 1
Priority: u=0, i
    
```

Summary: 22 requests, 2.98 MB / 44.74 kB transferred, Finish: 900 ms, DOMContentLoaded: 440 ms, load: 882 ms

HTTP Methods

- Web clients can use various request types when accessing a web server. These request types are called “request methods”.
 - GET
 - POST
 - HEAD
 - TRACE
 - OPTIONS
 - PUT
 - DELETE

GET Method

- Used to retrieve data
- Data is sent in URL
- Should not modify server data
- Example
GET /products?id=10 HTTP/1.1
- Features:
 - Cacheable
 - Bookmarkable
 - Limited data size

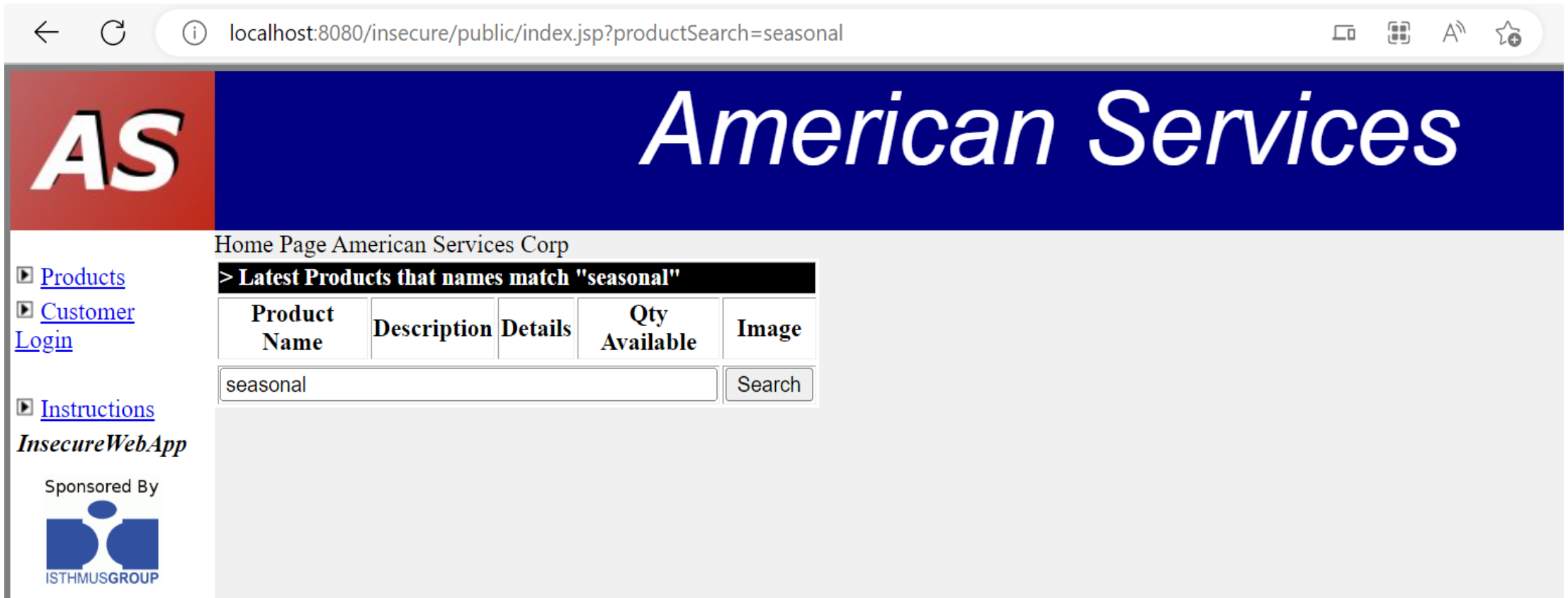
GET is used for:

- Viewing web pages.
- Searching information.
- Retrieving records from databases.

GET Method

- Get method is mainly used at the client (Browser) side to send a request to a specified server to get certain data or resources.
- Using this method the server should only let us receive the data and not change its state - it is only used to view something and not to change it.
- The request parameter of the get method is appended to the URL:
 - <http://abc.com/regis.php?name=sanjay&email=sanjaya@gmail.com>
- Can be cached
- Remain in the browser history
- Can be bookmarked
- Should never be used when dealing with sensitive data
- Have length restrictions (255 characters)
- Are only used to request data (not modify)

Get Request



← ↻ ⓘ localhost:8080/insecure/public/index.jsp?productSearch=seasonal



American Services

Home Page American Services Corp

▶ [Products](#)
▶ [Customer Login](#)
▶ [Instructions](#)

InsecureWeb.App

Sponsored By



> Latest Products that names match "seasonal"

Product Name	Description	Details	Qty Available	Image
seasonal			Search	

Post Method

- Post method is mainly used at the client side to send data to a specified server in order to create or rewrite a particular resource/data.
- This data sent to the server is stored in the request body of the HTTP request. Post method eventually leads to the creation of a new resource or updating an existing one.
- It is one of the most secure methods because the data that is been sent is included in the body of the request and not in the URL.
- Are never cached
- Do not remain in the browser history
- Cannot be bookmarked
- Have no restrictions on data length

POST Method

- Used to send data to server
- Data is sent in request body
- Used in forms and login systems

POST /login HTTP/1.1

- Features:
 - Not cached
 - More secure than GET for sensitive data
- POST is used for:
 - Sending data to the server
 - Submitting forms
 - Login requests
 - Uploading data

PUT Method

- Used to update or replace an existing resource.
- Sends complete updated data to the server.
- **Example:**
PUT /users/1 HTTP/1.1
- **Features:**
 - Idempotent method.
 - Replaces the entire resource.
 - Used for full updates.
- **PUT is used for:**
 - Updating user profiles.
 - Replacing existing records.
 - Updating database entries.
 - The PUT allows the client to upload files to the server.
 - This method can be dangerous to leave OPEN by web servers, as an attacker can easily upload malicious scripts to web servers and make have their attack to succeed.

DELETE Method

- Used to remove a resource from the server.
- **Example:**
DELETE /users/1 HTTP/1.1
- **Features:**
 - Idempotent method.
 - Removes specified data.
 - Common in REST APIs.
 - The DELETE method can be used to remove files in the web server through defined URLs.
- **DELETE is used for:**
 - Deleting user accounts.
 - Removing records.
 - Deleting files and resources.
 - This method is also unsafe to leave OPEN as this can cause of Denial of Service attack by deleting important files from web root directories and sometimes one can change the configuration of servers.

PUT vs POST Methods

Feature	POST	PUT
Purpose	Create a new resource	Update or replace an existing resource
Data Location	Request body	Request body
Idempotent	No	Yes
Resource URL	Usually server decides URL	Client specifies URL
Repeated Requests	May create multiple resources	Produces the same result each time
Common Use	Registration, Login, Form Submission	Updating user profile, Replacing records

Head Method

- The HEAD method can be used to view the Header information only and thus save bandwidth without sending the actual payload.
- This method is mostly used by developers and testers to see how much of server related information is revealed through header which the attacker can also use for his investigation/ information gathering process.
- identical to GET, but without the response body.
- **HEAD is used for:**
 - Checking file size.
 - Verifying resource availability.
 - Viewing metadata.

OPTIONS Method

- Used to discover supported HTTP methods.
- Helps clients understand server capabilities.
- **Example:**
OPTIONS /users HTTP/1.1
- **Features:**
 - Returns allowed methods.
 - Improves API communication.
- **OPTIONS is used for:**
 - API testing.
 - Checking server-supported methods.

How to check Which Method is allowed?

- Curl command
 - A command line tool that enables data exchange between a device and a server through a terminal
- The OPTIONS method lists out all the methods that are enabled by the web server.

```
root@kali:~# curl -v -X OPTIONS http://10.182.3.252
* Expire in 0 ms for 6 (transfer 0x561e69eb4dd0)
*   Trying 10.182.3.252...
* TCP_NODELAY set
* Expire in 200 ms for 4 (transfer 0x561e69eb4dd0)
* Connected to 10.182.3.252 (10.182.3.252) port 80 (#0)
> OPTIONS / HTTP/1.1
> Host: 10.182.3.252
> User-Agent: curl/7.64.0
> Accept: */*
<
< HTTP/1.1 200 OK
< Date: Thu, 19 Jan 2023 11:58:23 GMT
< Server: Apache/2.4.37 (centos)
< Allow: OPTIONS,HEAD,GET,POST,TRACE
< Content-Length: 0
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host 10.182.3.252 left intact
root@kali:~#
```


TRACE Method

- TRACE echoes the request as seen by the server back to the client
- –Allows attacker to see any changes made by intermediate servers, such as proxy

```
root@kali:~# curl -v -X TRACE http://10.182.3.252
* Expire in 0 ms for 6 (transfer 0x564dcf74add0)
*   Trying 10.182.3.252...
* TCP_NODELAY set
* Expire in 200 ms for 4 (transfer 0x564dcf74add0)
* Connected to 10.182.3.252 (10.182.3.252) port 80 (#0)
> TRACE / HTTP/1.1
> Host: 10.182.3.252
> User-Agent: curl/7.64.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Thu, 19 Jan 2023 16:39:20 GMT
< Server: Apache/2.4.37 (centos)
< Transfer-Encoding: chunked
< Content-Type: message/http
<
TRACE / HTTP/1.1
Host: 10.182.3.252
User-Agent: curl/7.64.0
Accept: */*

* Connection #0 to host 10.182.3.252 left intact
root@kali:~#
```

LAB Exercise – 10 Min

- Open web browser and press F12 to view developer mode
- Open following website : moodle.blr1.cdac.in
- Go to developer mode and click on network tab
 - View get request (take a screenshot and understand)
 - What are the various headers in the request?
 - What is the payload in the request?
 - View response (take a screenshot and understand)
 - What are various headers in the response?
 - What is the response code?
 - What is the payload in the response?
- Now Give username and password on the login fields
 - View request for this action
 - What is the type of the request?

HTTP Evolution

- Developed by Tim Berners-Lee in 1991, and is the underlying protocol of the World Wide Web.
- HTTP/0.9 – The one-line protocol
- HTTP/1.0 – Building extensibility
- HTTP/1.1 - The standardized protocol
- HTTP/2
- HTTP/3

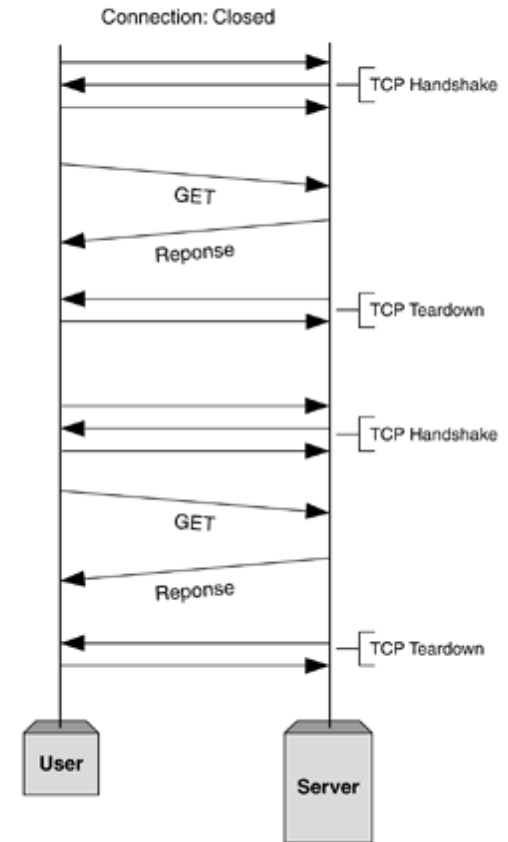
HTTP/0.9 (1991)

- **First Version of HTTP** - It was called as a the one-line protocol.
- Connection terminates immediately after the response.
- **Features**
 - Extremely simple protocol
 - Supported only GET requests
 - No headers
 - Response contained only HTML content
- **Example**
 - GET /index.html
- **Limitations**
 - No status codes
 - No content type information
 - No error handling
 - Only plain HTML documents

HTTP/1.0 (1996) – RFC 1945

• Features

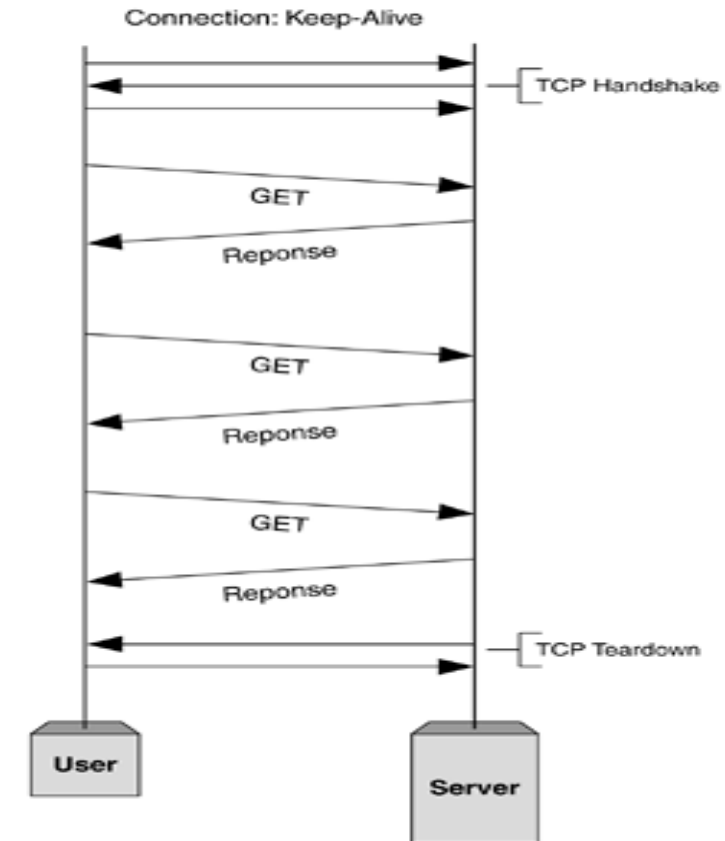
- Introduced request headers
- Introduced response headers
- Status codes added (200, 404, etc.)
- Support for different content types
- If the website is hosted multiple resources (IMG file, HTML etc), then multiple connection needs to be opened to get each resources.
- Methods supported: GET , HEAD , POST
- Client Request
 - GET /mypage.html HTTP/1.0
 - User-Agent: Mozilla/5.0 (Windows NT)
- Server Response
 - 200 OK
 - Date: Wed, 11 JAN 2023 11:25:11 IST
 - Server: CERN/3.0 libwww/2.17
 - Content-Type: text/html
 - <HTML>
 - The sample web page with image
 -
 - </HTML>



In a default HTTP/1.0 session, the TCP connection will be torn down and re-established between each HTTP GET request.

HTTP/1.1 (1997)

- **Most Widely Used Version**
- **New Features**
- **Persistent Connections (Keep-Alive)**
 - Reuse a single TCP connection
 - Reduced connection overhead
- **Host Header**
 - Multiple websites on one server
- **Chunked Transfer Encoding**
 - Send data in chunks
- **Additional Methods**
 - PUT
 - DELETE
 - OPTIONS
- **Benefits**
 - Faster page loading
 - Reduced network traffic



In a default HTTP/1.1 session, a single TCP connection will be held, open and multiple GET requests will be passed across.

HTTP/1.1 – RFC 2068

- Because of the Host header, it was possible to host several domains from a single IP address.
- Content negotiation, including language, encoding, and type, was added. A client and a server could now agree on which content to exchange.
- Revised two times – RFC 2616 and RFC 7230
- Used for more than 15 years till 2013

```
GET / HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:108.0) Gecko/20100101
Firefox/108.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/
webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
```

```
HTTP/1.1 200
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Date: Wed, 11 Jan 2023 05:26:47 GMT
Keep-Alive: timeout=20
Connection: keep-alive
```


Problems with HTTP/1.1

Performance Challenges

- **Head-of-Line Blocking**
 - Requests processed sequentially
- **Multiple Connections**
 - Browsers open many TCP connections
- **Increased Latency**
 - More network overhead

Modern Web Complexity

- Pages contain:
 - Images
 - CSS
 - JavaScript
 - Videos
- Result: Need for a faster protocol

- **1. Head-of-Line (HOL) Blocking**
- A TCP connection can process requests in order.
- If one request takes a long time, subsequent requests are delayed.
- This creates a bottleneck when loading many resources.
- **Example:**
A webpage requests:
 1. style.css
 2. logo.png
 3. app.js
- If style.css is delayed, the browser may wait before processing the next responses on that connection.

Problems with HTTP/1.1

Performance Challenges

- **Head-of-Line Blocking**
 - Requests processed sequentially
- **Multiple Connections**
 - Browsers open many TCP connections
- **Increased Latency**
 - More network overhead

Modern Web Complexity

- Pages contain:
 - Images
 - CSS
 - JavaScript
 - Videos
- Result: Need for a faster protocol

2. Multiple TCP Connections

- Browsers need many resources (images, CSS, JavaScript, fonts).
- HTTP/1.1 cannot efficiently handle many parallel requests on a single connection.
- Browsers open multiple TCP connections (often 6–8 per domain).
- **Problems:**
 - More memory usage
 - More CPU overhead
 - More network congestion

Problems with HTTP/1.1

Performance Challenges

- **Head-of-Line Blocking**
 - Requests processed sequentially
- **Multiple Connections**
 - Browsers open many TCP connections
- **Increased Latency**
 - More network overhead

Modern Web Complexity

- Pages contain:
 - Images
 - CSS
 - JavaScript
 - Videos
- Result: Need for a faster protocol

• 3. High Latency from TCP Handshakes

- Each new TCP connection requires:

1. SYN

2. SYN-ACK

3. ACK

- This is called the **TCP three-way handshake**.
- For secure HTTPS connections, an additional TLS handshake is required.
- **Result:**
 - Increased page load time
 - Poor performance on high-latency networks

Problems with HTTP/1.1

Performance Challenges

- **Head-of-Line Blocking**
 - Requests processed sequentially
- **Multiple Connections**
 - Browsers open many TCP connections
- **Increased Latency**
 - More network overhead

Modern Web Complexity

- Pages contain:
 - Images
 - CSS
 - JavaScript
 - Videos
- Result: Need for a faster protocol

4. Redundant Headers

- Every request sends headers such as:
 - Host: example.com
 - User-Agent: Chrome
 - Accept: */*
 - Cookie: ...
- These headers are often repeated for every resource request.
- **Problem:**
 - Wastes bandwidth
 - Increases network traffic
- HTTP/2 later solved this using **HPACK header compression**.

HTTP/2 – RFC 7540

- Called as a protocol for the greater performance.
- Server Push-ability for server to push resources to client without client sending a request explicitly.
- Headers are compressed. This reduces the duplication and overhead of data delivered because these are frequently identical within a group of queries.
- The limitations of the HTTP/1.x protocol can be overcome by making several requests over the same connection.
- Request and response multiplexing -allows the client and server to break down an HTTP message into independent frames, interleave them, and then reassemble them on the other end.
- –websockets, rest apis, web hooks
- Till Dec 2022 around 40% of internet websites were started using HTTP/2.
 - <https://w3techs.com/technologies/details/ce-http2>

HTTP/3 – RFC 9114

- It uses QUIC (Quick UDP Internet Connections) instead of TCP at Transport Layer
- QUIC is a multiplexed transport protocol implemented on UDP.
- QUIC is intended to significantly reduce HTTP connection latency.
- HTTP/3 is used by 25% of Internet websites as of Dec 2022.
 - <https://w3techs.com/technologies/details/ce-http3>
- **Why QUIC?**
 - Faster connection establishment
 - Better handling of packet loss
 - Improved performance on mobile networks

Web Cookies

- Cookies are small text files stored on a user's device by websites.
- They help websites remember information about users.
- Commonly used to improve browsing experience.
- **How Cookies Work**
 1. User visits a website.
 2. Website sends a cookie to the browser.
 3. Browser stores the cookie.
 4. Cookie is sent back to the website on future visits.

Cookies

- A cookie, also known as an HTTP cookie, web cookie, Internet cookie, or browser cookie, is a small piece of data sent from a website and stored in a user's web browser while the user is browsing that website. Every time the user loads the website, the browser sends the cookie back to the server to notify the website of the user's previous activity.
- Cookies are generally used for Session Management, Personalization and Tracking.

Session Hijacking Using Cookie Stealing

- Session hijacking is an attack where an attacker takes control of a user's active web session.
- Cookie stealing is one of the most common methods used to perform session hijacking.
- The attacker gains unauthorized access without knowing the user's password.

What Are Cookies?

- Small pieces of data stored by a web browser.
- Used to maintain user sessions after login.
- Common cookie types:
 - Session cookies
 - Persistent cookies
 - Authentication cookies

How Web Sessions Work

1. User logs in.
2. Server validates credentials.
3. Server creates a session.
4. Session ID is stored in a cookie.
5. Browser sends the cookie with future requests.
6. Server recognizes the user through the session ID.

What is Cookie Stealing?

- Unauthorized acquisition of a user's session cookie.
- If the stolen cookie remains valid, an attacker may impersonate the user.
- Does not necessarily require the user's password.

Common Causes of Cookie Theft

- Cross-Site Scripting (XSS)
- Malware on the victim's device
- Insecure Wi-Fi networks
- Unencrypted HTTP connections
- Browser extensions with excessive permissions
- Physical access to a logged-in device

Session Hijacking Process

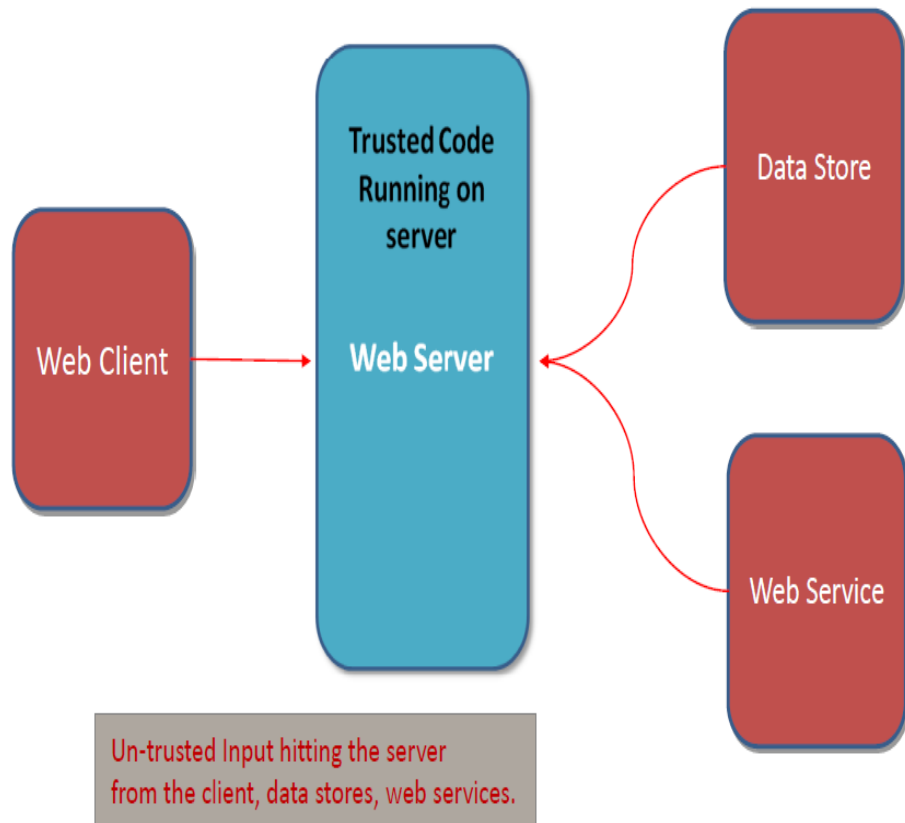
- **General Overview**

1. User authenticates.
2. Session cookie is created.
3. Attacker obtains the cookie.
4. Attacker uses the stolen session token.
5. Server treats attacker as the legitimate user.

Web Application Security

- Is a branch of Information Security that deals specifically with security of websites, web applications and web services.
- At a high level, Web application security draws on the principles of application security but applies them specifically to Internet and Web systems.
- Typically web applications are developed using programming languages such as PHP , Java EE , Java , Python , Ruby , ASP.NET , C#, VB.NET or Classic ASP.

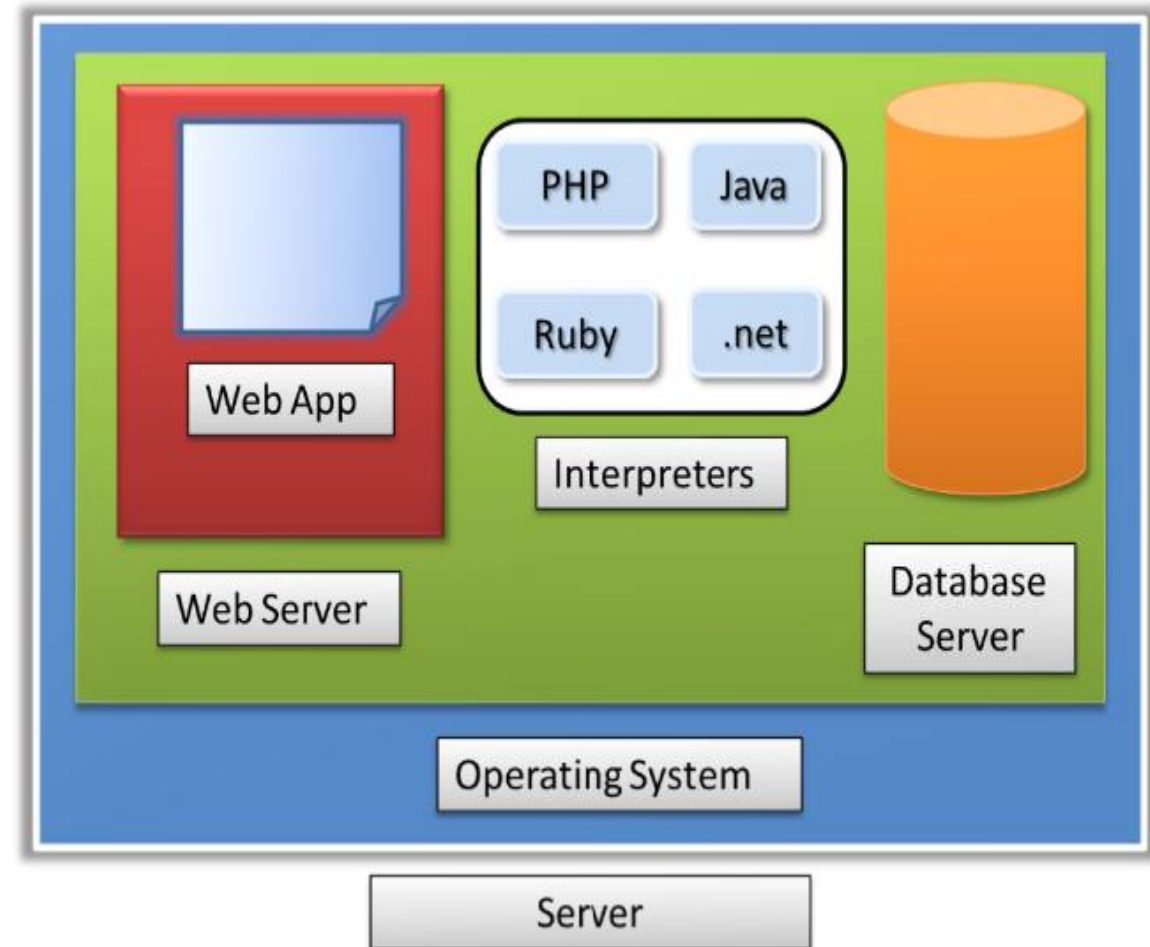
Architecture of Web Application



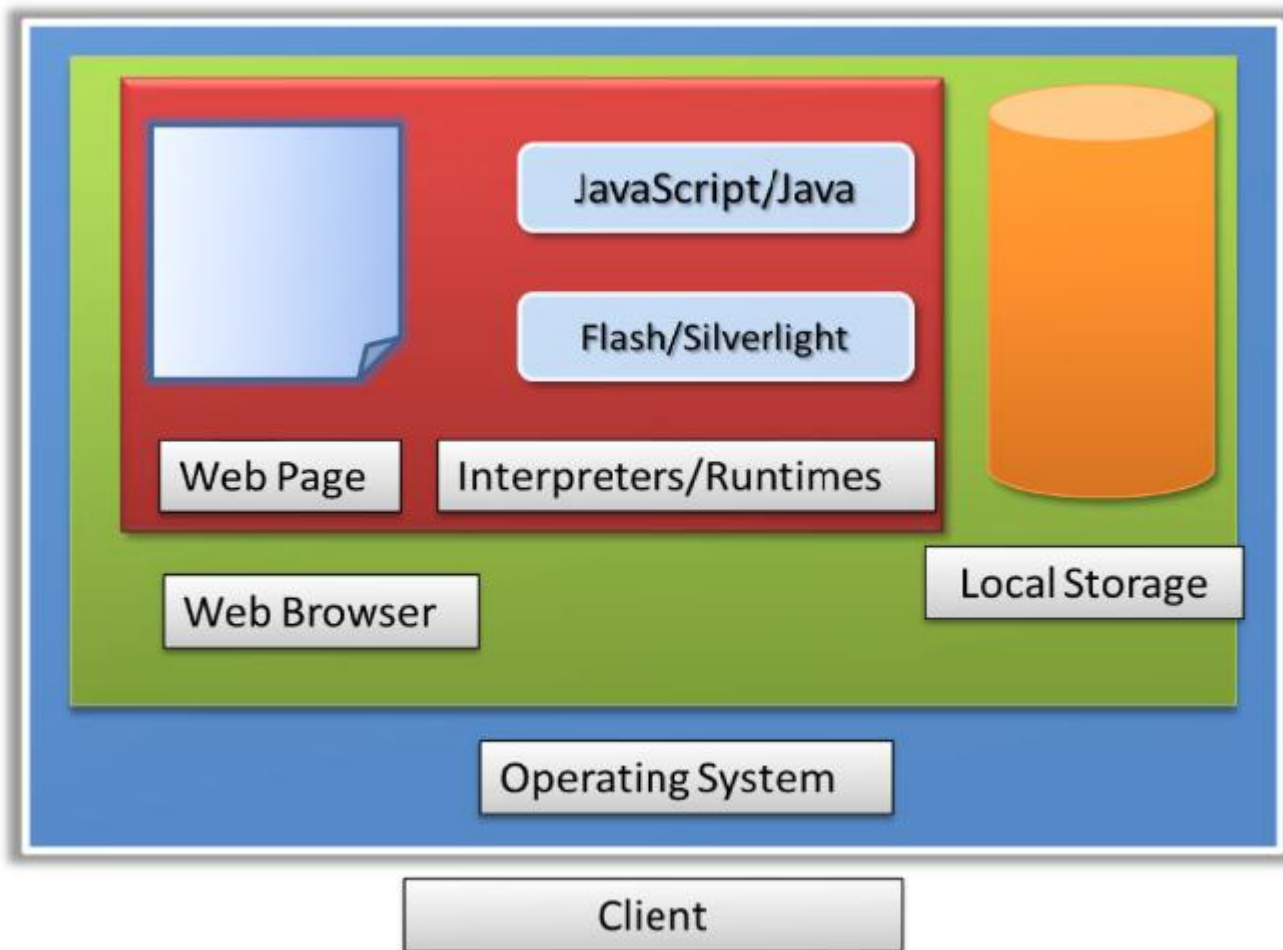
- The only piece we can *trust* is the *code* running on the server.
- Any data coming to the server, regardless of where it originated from shouldn't be trusted.
- Data can only be trusted once it has been validated by some piece of *trusted code*.

Web Server Architecture

- The important thing to understand is that a pure web server ONLY grabs files and sends them along to the browser.
- If you want the server to do something more like retrieve information from a database or do some processing, you need an application server.



Web Client Architecture

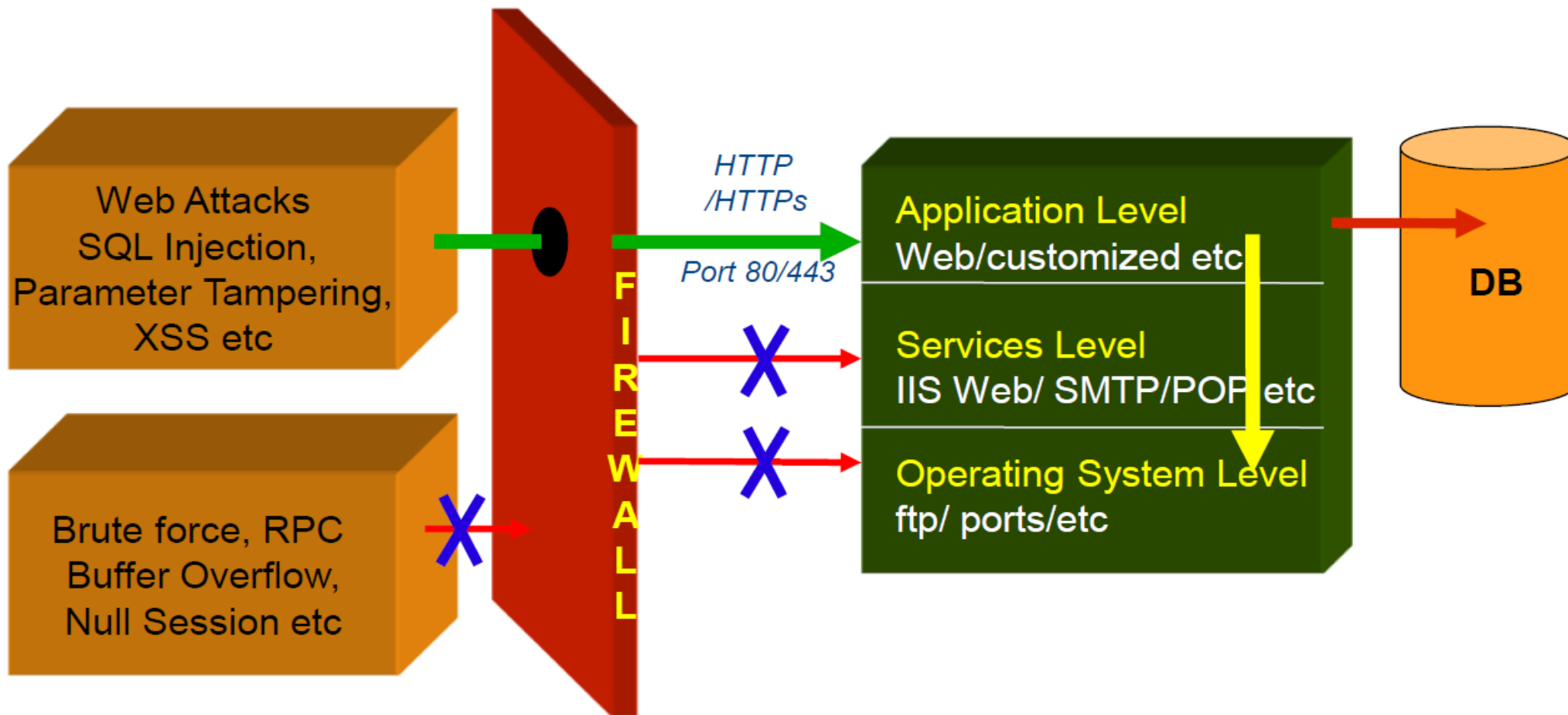


- Web App Client is a nothing but a web browser , it is a software application program used to view, retrieve, present or upload, surf, traverse information resources like documents or html pages which consists of text, graphics or links on the World Wide Web(internet).

Client and Server Side Processing

- Client-side processing
 - Some processing needs to be “executed” by the browser, either to form the request for dynamic web page or to create or display the web page.
 - Eg: javascript code to validate the user input
- Server-Side processing
 - In server side processing, the web server
 - Receives the dynamic web page request
 - Performs all of the processing necessary to create the dynamic web page
 - Sends the finished Web page to the client for display in the client’s browser

Scenarios of Web Attacks



Introduction to Proxy Server



Proxy Server

- In computer networks, a proxy server is a server (a computer system or an application program) that services the requests of its clients by forwarding requests to other servers.
- A client connects to the proxy server, requesting some service, such as a file, connection, web page, or other resource, available from a different server. The proxy server provides the resource by connecting to the specified server and requesting the service on behalf of the client.
- A proxy server may optionally alter the client's request or the server's response, and sometimes it may serve the request without contacting the specified server. In this case, it would 'cache' the first request to the remote server, so it could save the information for later, and make everything as fast as possible

Introduction to Burp Suite

- A web application security testing platform used by security professionals and penetration testers.
- **Capabilities**
 - Intercept web traffic
 - Analyze requests and responses
 - Discover vulnerabilities
 - Perform security assessments

Burpsuite Proxy

- The Proxy tool lies at the heart of Burp's user-driven workflow, and gives you a direct view into how your target application works "under the hood". It operates as a web proxy server, and sits as a man-in-the-middle between your browser and destination web servers. This lets you intercept, inspect, and modify the raw traffic passing in both directions.
- If the application employs HTTPS, Burp breaks the TLS connection between your browser and the server, so that even encrypted data can be viewed and modified within Burp's tools.

Burpsuite Demo

- Download and Install Burpsuit
 - <https://portswigger.net/burp/communitydownload>
- Open Burpsuite
- TAB -Proxy
 - Options
 - Proxy Listeners -Edit host and port
 - Intercept
 - Intercept is on/off
- Access Browser
 - Set your browser settings to listen to burp proxy
 - Proxy -change host and port
- Start accessing your target application

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Discover

Tasks

New scan

New live task

Filter

Search

1. Live passive crawl from Proxy (all traffic)

Add links. Add item itself, same domain and URLs in suite scope.

Capturing

1. Live passive crawl from Proxy (all traffic)

Summary

Items added to site map

Host	Method	URL	Status code	MIME type
moodle.blr1.cdac.in	GET	/	303	HTML
moodle.blr1.cdac.in	GET	/login/index.php	200	HTML
moodle.blr1.cdac.in	GET	/theme/image.php/clean/theme/176303...	200	
moodle.blr1.cdac.in	GET	/theme/yui_combo.php?rollup/3.17.2/yui...	200	CSS
moodle.blr1.cdac.in	GET	/theme/styles.php/clean/1763037736/all	200	CSS
moodle.blr1.cdac.in	GET	/theme/yui_combo.php?rollup/3.17.2/yui...	200	script
moodle.blr1.cdac.in	GET	/lib/javascript.php/1763037736/lib/javas...	200	script
moodle.blr1.cdac.in		/help.php?component=moodle&iidentifi...		
moodle.blr1.cdac.in	GET	/theme/image.php/clean/core/1763037...	200	XML
moodle.blr1.cdac.in		/lib/requirejs.php/1763037736/		
moodle.blr1.cdac.in		/lib/javascript.php/1763037736/lib/jquer...		
moodle.blr1.cdac.in		/lib/javascript.php/1763037736/lib/jquer...		
moodle.blr1.cdac.in		/lib/javascript.php/1763037736/lib/requi...		
moodle.blr1.cdac.in	GET	/lib/javascript.php/1763037736/lib/requi...	200	script
moodle.blr1.cdac.in	GET	/theme/javascript.php/clean/176303773...	200	script
moodle.blr1.cdac.in		/login/forgot_password.php		
moodle.blr1.cdac.in		/assets/skins/sam/yui2-skin		
moodle.blr1.cdac.in	POST	/login/index.php		
moodle.blr1.cdac.in	POST	/login/index.php		
moodle.blr1.cdac.in	GET	/theme/yui_combo.php?3.17.2/cssbutto...	200	CSS
moodle.blr1.cdac.in	GET	/theme/yui_combo.php?3.17.2/plugin/pl...	200	script
moodle.blr1.cdac.in	GET	/pluginfile.php/1/theme_clean/logo/176...	200	XML
moodle.blr1.cdac.in	GET	/lib/requirejs.php/1763037736/core/first...	200	script
moodle.blr1.cdac.in	GET	/lib/javascript.php/1763037736/lib/requi...	200	script
moodle.blr1.cdac.in	GET	/lib/javascript.php/1763037736/lib/jquer...	200	script
moodle.blr1.cdac.in	POST	/lib/ajax/service.php?sesskey=mLBxroF...	200	JSON
moodle.blr1.cdac.in	POST	/login/index.php	303	HTML
moodle.blr1.cdac.in	GET	/theme/image.php/clean/core/1763037...	200	XML

Task configuration

Task type: Live passive crawl

Scope: Proxy (all traffic)

Configuration: Add links. Add item itself, same domain and URLs in suite scope.

Capturing

Task progress

Site map items added: 28

Responses processed: 22

Responses queued: 0

Task log

> Adding link http://moodle.blr1.cdac.in/theme/image.php/clean/core/1763037736/i/warning in response http://mood le.blr1.cdac.in/login/index.php to site map

> Adding response http://moodle.blr1.cdac.in/login/index.php to site map

> Adding response http://moodle.blr1.cdac.in/lib/ajax/service.php?sesskey=mLBxroFGZI to site map

> Adding response http://moodle.blr1.cdac.in/lib/javascript.php/1763037736/lib/jquery/jquery-1.12.1.min.js to site ma p

> Adding response http://moodle.blr1.cdac.in/lib/javascript.php/1763037736/lib/requirejs/jquery-private.js to site ma p

> Adding response http://moodle.blr1.cdac.in/lib/requirejs.php/1763037736/core/first.js to site map

> Adding response http://moodle.blr1.cdac.in/pluginfile.php/1/theme_clean/logo/1763037736/CDAC.svg to site map

> Adding response http://moodle.blr1.cdac.in/theme/yui_combo.php?3.17.2/plugin/plugin-min.js&m/1763037736/core /lockscroll/lockscroll-min.js to site map

> Adding response http://moodle.blr1.cdac.in/theme/yui_combo.php?3.17.2/cssbutton/cssbutton-min.css to site map

> Adding form http://moodle.blr1.cdac.in/login/index.php in response http://moodle.blr1.cdac.in/login/index.php to si te map

> Adding form http://moodle.blr1.cdac.in/login/index.php in response http://moodle.blr1.cdac.in/login/index.php to si te map

Event log (2)

All issues

Memory: 198.0MB of 15.84GB

Disabled

Dashboard

- The Dashboard is the home screen.
- Purpose:
 - Shows the status of scans and tasks.
 - Displays discovered issues.
 - Monitors Burp's activity and event logs.
 - Provides shortcuts to common actions.
- What you will see:
 - Tasks: Running scans and crawls.
 - Issue Activity: Vulnerabilities found.
 - Event Log: Warnings, errors, and status messages.
 - Live Tasks: Current operations.
- Use it to monitor everything Burp is doing.

Target

- **Site Map**
- Displays all discovered:
 - URLs
 - Directories
 - API endpoints
 - Files
- **Scope**
 - Defines which websites Burp should focus on.
- **Benefits:**
 - Reduces noise.
 - Prevents accidental testing of unrelated sites.

Proxy

- One of Burp's most important tools.
- **Intercept**
 - Acts as a man-in-the-middle proxy between browser and server.
 - You can:
 - View requests
 - Modify requests
 - Forward requests
 - Drop requests
- **HTTP History**
 - Stores all traffic passing through Burp.
 - Useful for:
 - Reviewing requests
 - Finding endpoints
 - Sending requests to other tools

Intruder

- Used for automated attacks and fuzzing.
- **Purpose**
- Tests:
 - Parameters
 - Login forms
 - APIs
 - Hidden values

Intruder Workflow

- Intruder has four main sections:
 1. Target
 2. Positions
 3. Payloads
 4. Settings

```
graph TD; A[Capture Request] --> B[Send to Intruder]; B --> C[Select Positions]; C --> D[Configure Payloads]; D --> E[Start Attack]; E --> F[Analyze Results];
```

1. Target

- This specifies where requests will be sent.
- Example:
 - POST /login HTTP/1.1
Host: example.com
- Intruder automatically extracts:
 - Host: example.com
Port: 443
Protocol: HTTPS
- You usually don't need to change this.

2. Positions

- This is the most important tab.
- It tells Intruder:
- "Which parts of the request should be modified?"
- **Example Request**
 - GET /profile?id=123 HTTP/1.1
Host: example.com
- Suppose you want to test different IDs.
- Mark the position:
 - GET /profile?id=§123§ HTTP/1.1
- The § symbols indicate payload positions.

2. Positions – cont.

- **Multiple Positions**

- Example:

- POST /login HTTP/1.1

username=\$admin&password=\$password\$

- Now Intruder can modify both fields.

- **Multiple Positions**

- Example:

- POST /login HTTP/1.1

username=\$admin&password=\$password\$

- Now Intruder can modify both fields.

Attack Types

- Intruder provides several attack modes.
- **A. Sniper**
- **B. Battering Ram**
- **C. Pitchfork**
- **D. Cluster Bomb**

A. Sniper

- Most commonly used.
- **How It Works**
- Tests one position at a time.
- Example:
 - GET /user?id=\$1\$
- Payload list:
 - 1
 - 2
 - 3
 - 4
 - 5
- Requests generated:
 - GET /user?id=1
 - GET /user?id=2
 - GET /user?id=3
 - GET /user?id=4
 - GET /user?id=5
- Best for:
 - Parameter testing
 - Resource enumeration
 - Input validation checks

B. Battering Ram

- Uses the same payload in every position.
- Positions:
 - username=\$admin\$
password=\$admin\$
- Payloads:
 - test
 - root
 - admin
- Generated:
 - username=test password=test
 - username=root password=root
 - username=admin password=admin
- Useful when multiple fields should contain the same value.

C. Pitchfork

- Uses multiple payload sets in parallel.
- Payload Set 1:
 - alice
bob
charlie
- Payload Set 2:
 - pass1
pass2
pass3
- Generated:
 - alice / pass1
bob / pass2
charlie / pass3
- Each row is paired.

D. Cluster Bomb

- Tests every combination.
- Payload Set 1:
 - alice
bob
- Payload Set 2:
 - 123
password
- Generated:
 - alice / 123
alice / password
bob / 123
bob / password
- Number of requests grows rapidly.
- Formula:
 - Total Requests =
PayloadSet1 × PayloadSet2 × PayloadSet3...

4. Payloads

- Payloads are the values Intruder inserts.

Repeater

- Manual request testing tool.
- **Purpose**
- Send the same request repeatedly with modifications.
- Example:
- Original:
 - GET /user?id=1
- Try:
 - GET /user?id=2
 - GET /user?id=3
 - GET /user?id='
- Observe:
 - Responses
 - Errors
 - Data leaks
- Very useful for:
 - SQL injection testing
 - IDOR testing
 - Parameter manipulation